

Sprint 4 — Gestão de Secrets em Pipeline DevSecOps

Nome: Samuel Castro dos Santos

Materia: Ciber criptografia aplicada

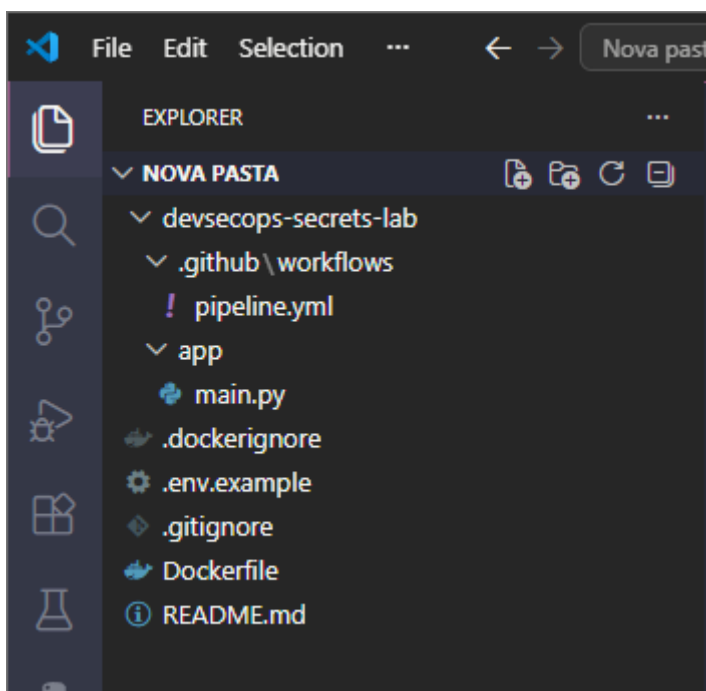
1. Objetivo

Demonstrar a identificação e mitigação de riscos relacionados ao vazamento de secrets em um ambiente DevSecOps utilizando GitHub Actions, Docker e variáveis de ambiente.

2. Estrutura do projeto

Foi criada a estrutura básica do projeto contendo a aplicação, workflow do GitHub Actions, arquivos de configuração e artefatos de segurança.

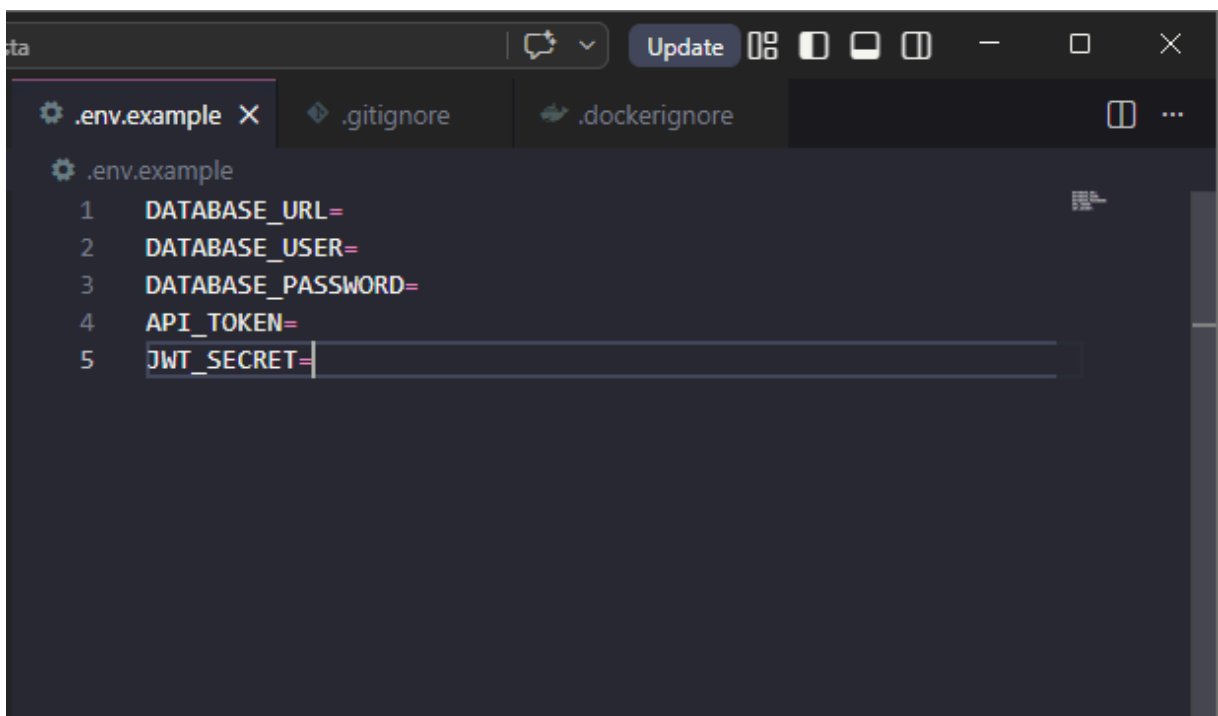
Figura 1 – Estrutura do projeto



3. Configuração do arquivo `.env.example`

O arquivo `.env.example` foi utilizado para demonstrar quais variáveis de ambiente são necessárias para a aplicação, sem expor valores reais.

Figura 2 – Arquivo `.env.example`



The image shows a code editor window with a dark theme. The title bar at the top includes a search icon, a dropdown menu, and an 'Update' button. Below the title bar, there are three tabs: `.env.example` (active), `.gitignore`, and `.dockerignore`. The `.env.example` file is open, showing five lines of environment variables:

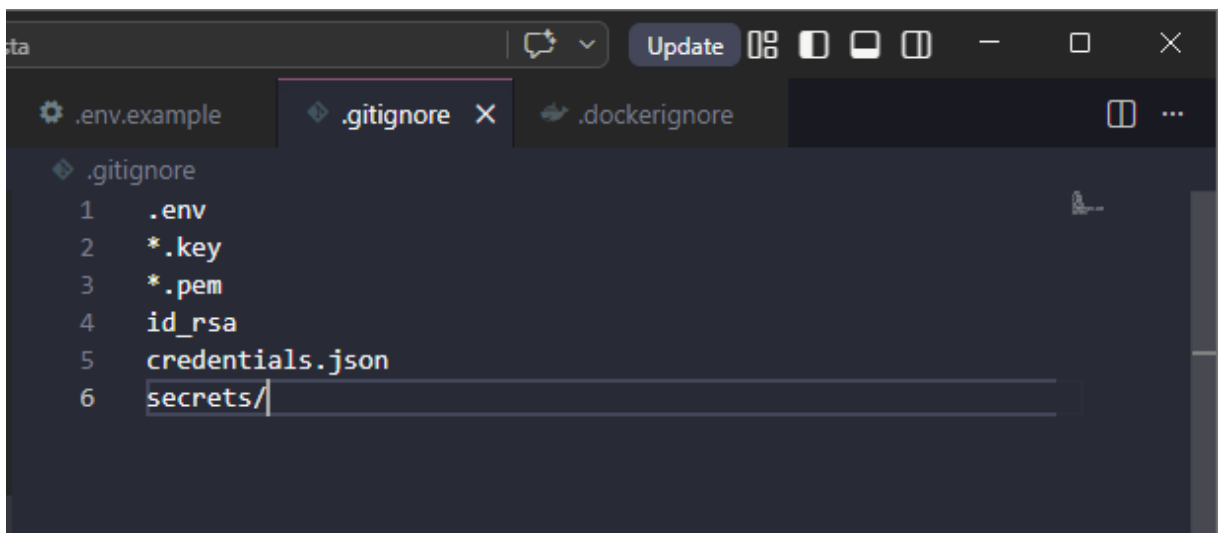
```
1 DATABASE_URL=  
2 DATABASE_USER=  
3 DATABASE_PASSWORD=  
4 API_TOKEN=  
5 JWT_SECRET=
```

The cursor is positioned at the end of the fifth line, `JWT_SECRET=`.

4. Proteção Contra versionamento de secrets

Para impedir o envio acidental de credenciais ao repositório Git, foi configurado o arquivo *.gitignore*.

Figura 3 – Arquivo *.gitignore*



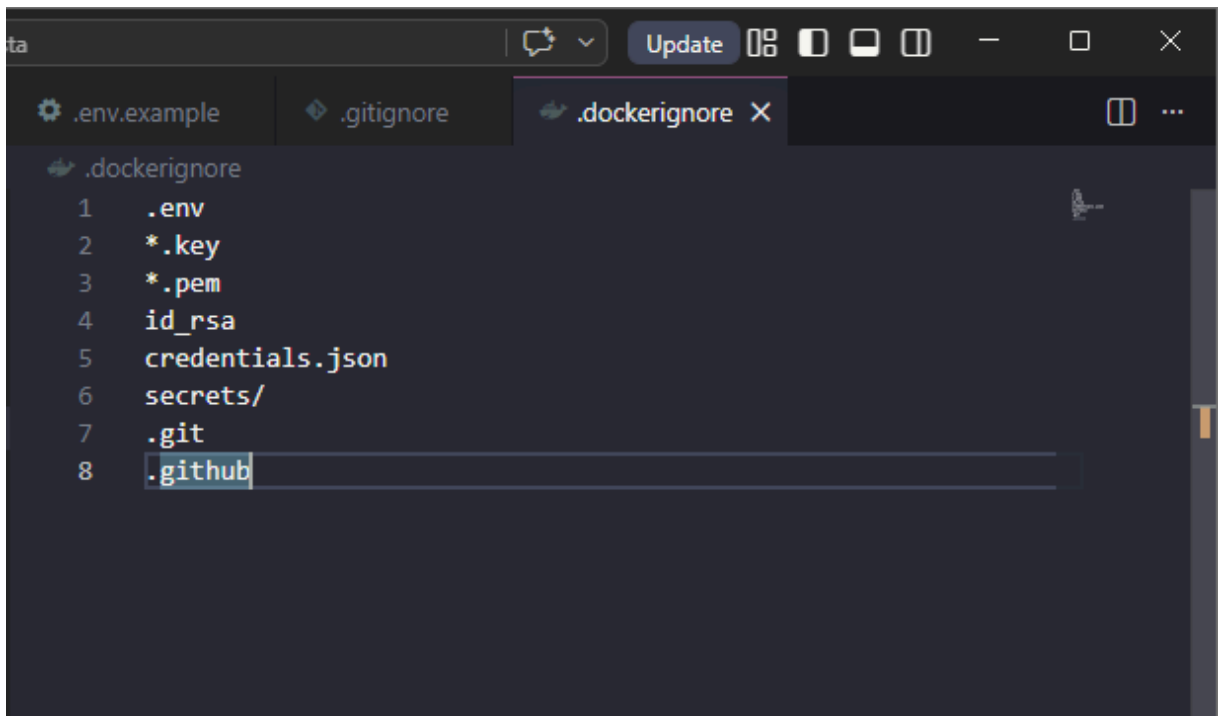
The image shows a code editor window with a dark theme. The title bar at the top includes a search icon, a dropdown menu, and an 'Update' button. Below the title bar, there are three tabs: '.env.example', '.gitignore' (which is active and highlighted), and '.dockerignore'. The main editing area shows the content of the '.gitignore' file, which is as follows:

```
1 .env
2 *.key
3 *.pem
4 id_rsa
5 credentials.json
6 secrets/
```

5. Proteção do contexto de build docker

Foi configurado o arquivo **.dockerignore** para impedir que arquivos sensíveis sejam enviados durante a construção da imagem Docker.

Figura 4 – Arquivo .dockerignore



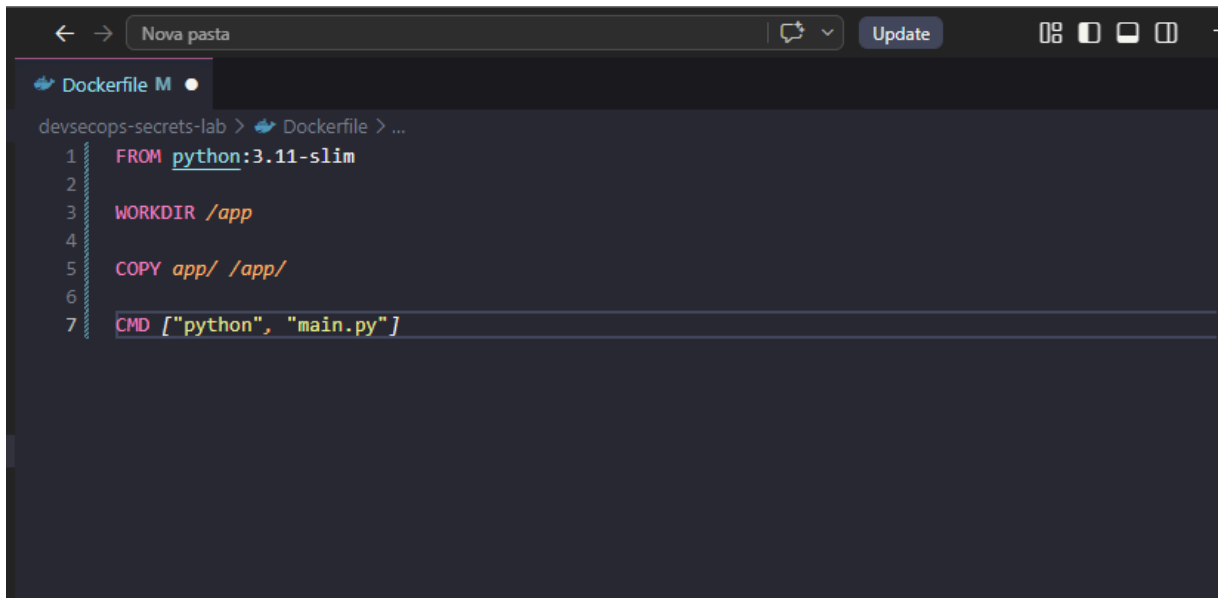
The image shows a code editor window with a dark theme. The title bar at the top includes a search icon, a dropdown menu, an 'Update' button, and standard window controls. The editor has three tabs: '.env.example', '.gitignore', and '.dockerignore' (which is active). The '.dockerignore' file contains the following text:

```
.dockerignore
1  .env
2  *.key
3  *.pem
4  id_rsa
5  credentials.json
6  secrets/
7  .git
8  .github
```

6. Dockerfile Seguro

O Dockerfile foi desenvolvido sem copiar arquivos contendo secrets para dentro da imagem.

Figura 5 – Dockerfile utilizado

A screenshot of a code editor showing a Dockerfile. The editor has a dark theme. At the top, there's a breadcrumb navigation bar showing 'devsecops-secrets-lab > Dockerfile > ...'. Below this, the Dockerfile content is displayed with line numbers 1 through 7 on the left. The code is as follows:

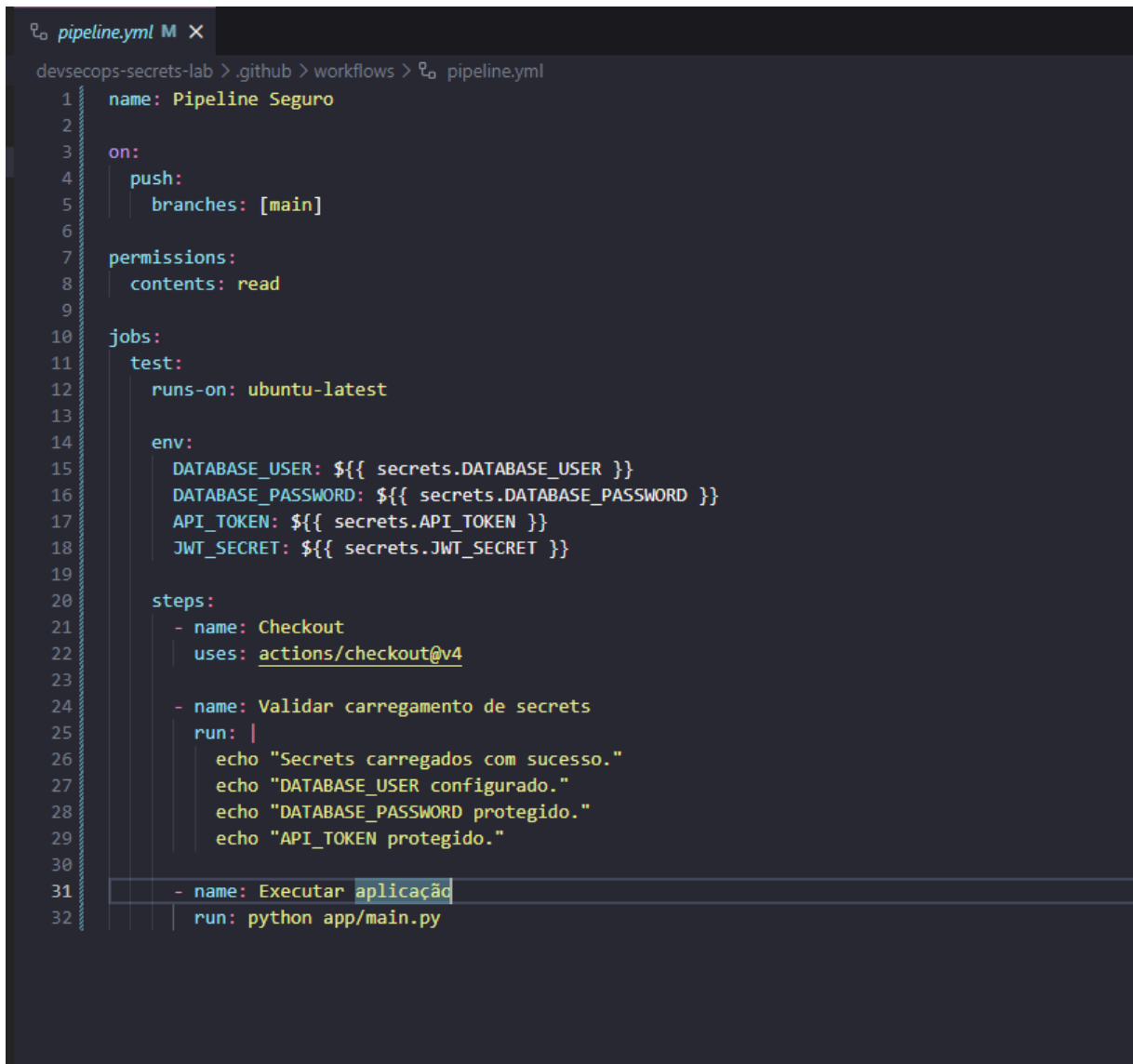
```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY app/ /app/
6
7 CMD ["python", "main.py"]
```

The code is syntax-highlighted: 'FROM' is in pink, 'python:3.11-slim' is in blue, 'WORKDIR' is in pink, '/app' is in orange, 'COPY' is in pink, 'app/' is in orange, and 'CMD' is in pink. The arguments in the CMD instruction are in quotes and color-coded. The editor interface includes a top bar with navigation arrows, a search bar labeled 'Nova pasta', and an 'Update' button. On the right side of the top bar, there are icons for file operations. A tab labeled 'Dockerfile M' is visible at the top left of the editor area.

7. Workflow Seguro do GitHub Actions

O pipeline foi configurado para utilizar GitHub Secrets e evitar a exposição de credenciais nos logs.

Figura 6 – Workflow pipeline.yml

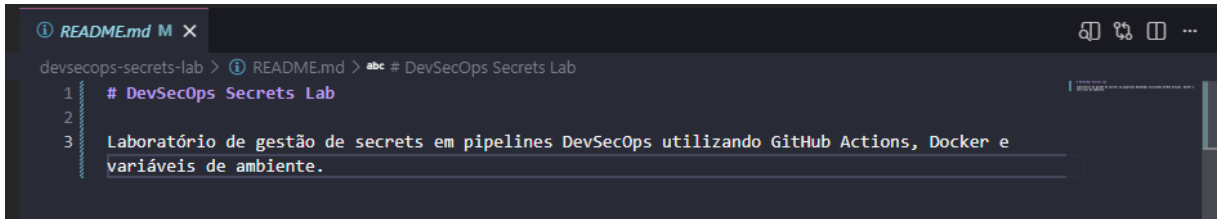


```
1  name: Pipeline Seguro
2
3  on:
4    push:
5      branches: [main]
6
7  permissions:
8    contents: read
9
10 jobs:
11   test:
12     runs-on: ubuntu-latest
13
14     env:
15       DATABASE_USER: ${ secrets.DATABASE_USER }
16       DATABASE_PASSWORD: ${ secrets.DATABASE_PASSWORD }
17       API_TOKEN: ${ secrets.API_TOKEN }
18       JWT_SECRET: ${ secrets.JWT_SECRET }
19
20     steps:
21       - name: Checkout
22         uses: actions/checkout@v4
23
24       - name: Validar carregamento de secrets
25         run: |
26           echo "Secrets carregados com sucesso."
27           echo "DATABASE_USER configurado."
28           echo "DATABASE_PASSWORD protegido."
29           echo "API_TOKEN protegido."
30
31       - name: Executar aplicação
32         run: python app/main.py
```

8. Documentação do projeto

Foi criado um arquivo README contendo a descrição básica do laboratório.

Figura 7 – README.md



```
devsecops-secrets-lab > @ README.md > abc # DevSecOps Secrets Lab
1 # DevSecOps Secrets Lab
2
3 Laboratório de gestão de secrets em pipelines DevSecOps utilizando GitHub Actions, Docker e
  variáveis de ambiente.
```

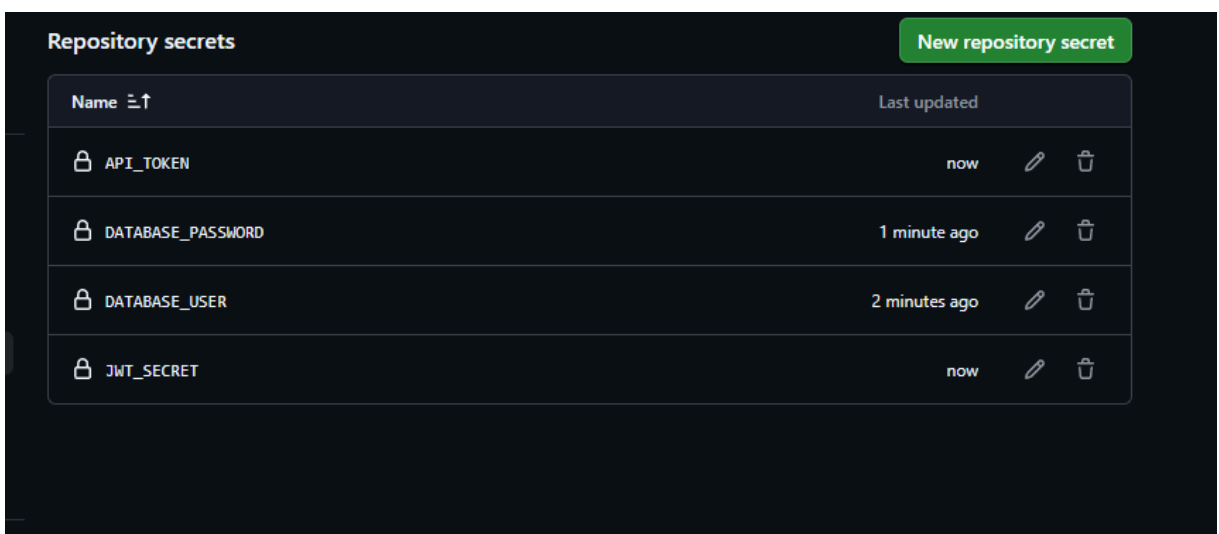
9. Configuração dos GitHub secrets

Foram cadastrados os seguintes secrets:

- ***DATABASE_USER***
- ***DATABASE_PASSWORD***
- ***API_TOKEN***
- ***JWT_SECRET***

Os valores permanecem ocultos e são utilizados apenas durante a execução do pipeline.

Figura 8 – GitHub Repository Secrets



Repository secrets		New repository secret	
Name	Last updated		
 API_TOKEN	now		
 DATABASE_PASSWORD	1 minute ago		
 DATABASE_USER	2 minutes ago		
 JWT_SECRET	now		

10. Execução do pipeline

Após o push para o repositório, o workflow foi executado automaticamente pelo GitHub Actions.

Figura 9 – Workflow executado

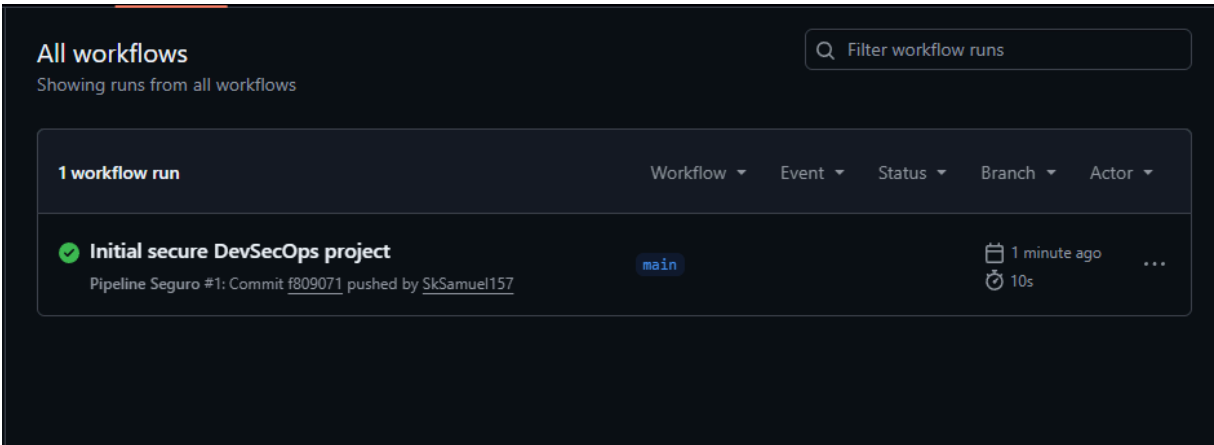
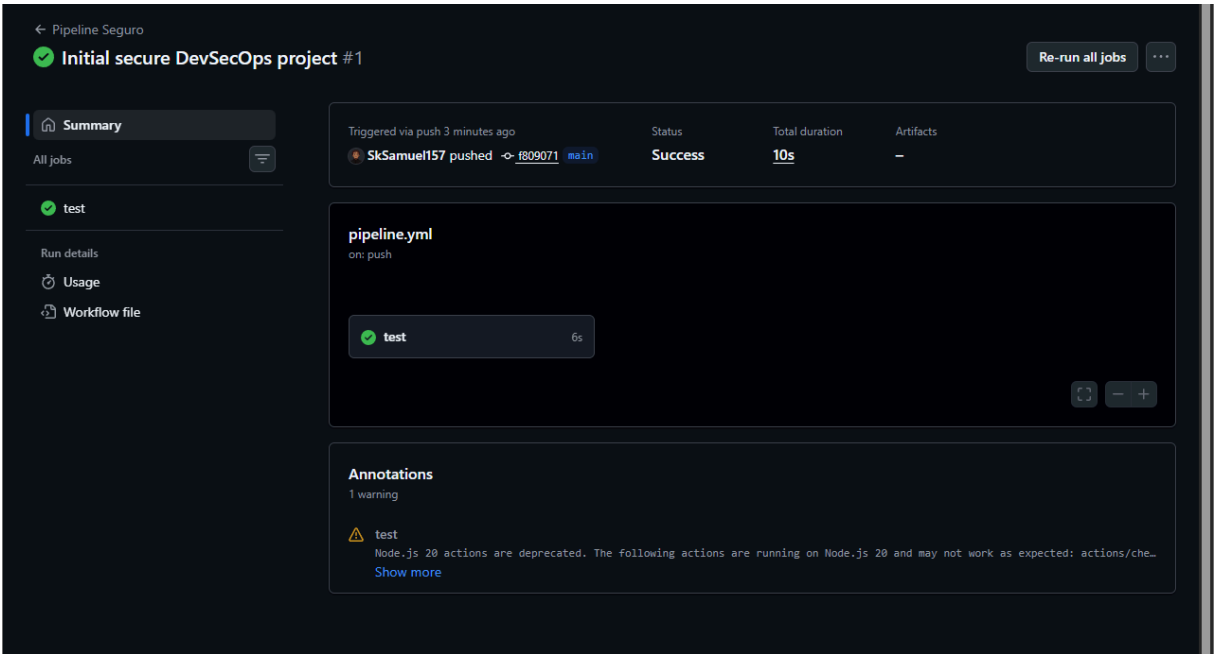


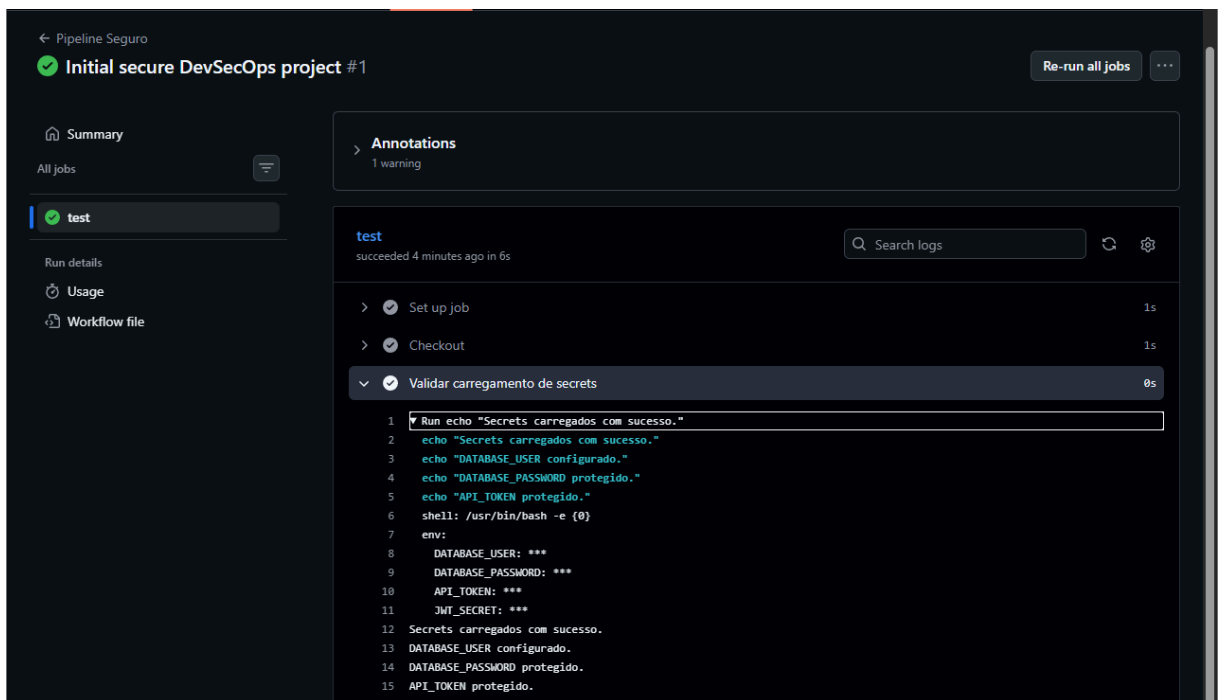
Figura 10 – Execução concluída com sucesso



11. Validação dos secrets no pipeline

Os logs demonstram que os *secrets* foram carregados corretamente sem exibir seus valores reais.

Figura 11 – Validação dos secrets



12. Build da imagem docker

Foi realizada a construção da imagem Docker da aplicação.

Comando utilizado:

docker build -t devsecops-secrets-lab .

Figura 12 – Build da imagem Docker

```
PS C:\Users\samuel\Desktop\Nova pasta\devsecops-secrets-lab> docker build -t devsecops-secrets-lab .
[+] Building 8.2s (9/9) FINISHED
=> [internal] load build definition from Dockerfile                                docker:desktop-linux 0.1s
=> => transferring dockerfile: 122B                                              0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim              2.5s
=> [auth] library/python:pull token for registry-1.docker.io                    0.0s
=> [internal] load .dockerignore                                                 0.1s
=> => transferring context: 109B                                                 0.0s
=> [1/3] FROM docker.io/library/python:3.11-slim@sha256:f9fa7f851e38bfb19c9de3afbc4b86ae7176ea7aaf94535c31df5458d5849457 4.1s
=> => resolve docker.io/library/python:3.11-slim@sha256:f9fa7f851e38bfb19c9de3afbc4b86ae7176ea7aaf94535c31df5458d5849457 0.1s
=> => sha256:3250ea7ceadc1e9a40a2d832082ee8fe68631ef9c9bf7bd5447fbcff9f527439 250B / 250B 0.2s
=> => sha256:8bd2cf7d3f6acc894d6ab00affeb6abf779933fa982aec49b9a4e322b7fbb 14.36MB / 14.36MB 1.1s
=> => sha256:8c471a93fc525f2012a1d483a12540ba9df9189f0bfe12238f74273aa72590a7 1.29MB / 1.29MB 0.5s
=> => sha256:72c03230f1363a3fb61d2f98504cf168bad3fe22f511ad2005dc021515d7ce97 29.79MB / 29.79MB 1.7s
=> => extracting sha256:72c03230f1363a3fb61d2f98504cf168bad3fe22f511ad2005dc021515d7ce97 1.0s
=> => extracting sha256:8c471a93fc525f2012a1d483a12540ba9df9189f0bfe12238f74273aa72590a7 0.1s
=> => extracting sha256:8bd2cf7d3f6acc894d6ab00affeb6abf779933fa982aec49b9a4e322b7fbb 0.7s
=> => extracting sha256:3250ea7ceadc1e9a40a2d832082ee8fe68631ef9c9bf7bd5447fbcff9f527439 0.0s
=> [internal] load build context                                                 0.1s
=> => transferring context: 615B                                                 0.0s
=> [2/3] WORKDIR /app                                                           0.2s
=> [3/3] COPY app/ /app/                                                         0.1s
=> exporting to image                                                            0.8s
=> => exporting layers                                                            0.4s
=> => exporting manifest sha256:28ce8bd51229b25f02accc3395acf8e9dbcae690b1b85e7b502beba503d2bb87 0.0s
=> => exporting config sha256:bcf05a3337dda262e9b7266c7c9a96805872fec382caff58d49431e0d01d3de4 0.0s
=> => exporting attestation manifest sha256:c98e3f2cc77f048fdb1db61e4a204348d66b030ec5679f0aee4cc189b1ae2828 0.1s
=> => exporting manifest list sha256:b77903632f0534cb3507750435348203937abffe85f8f224f1ddb957c54d7dcd 0.0s
=> => naming to docker.io/library/devsecops-secrets-lab:latest                 0.0s
=> => unpacking to docker.io/library/devsecops-secrets-lab:latest               0.1s
```

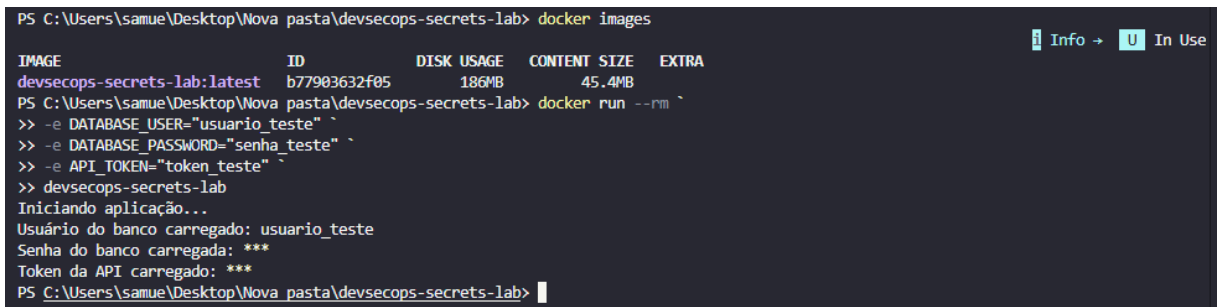
13. Validação da imagem e execução do container

A imagem criada foi validada e executada utilizando variáveis de ambiente fornecidas em tempo de execução.

Comandos utilizados:

```
docker images
docker run --rm -e DATABASE_USER="usuario_teste" -e
DATABASE_PASSWORD="senha_teste" -e API_TOKEN="token_teste"
devsecops-secrets-lab
```

Figura 13 – Imagem criada e execução do container



```
PS C:\Users\samue\Desktop\Nova pasta\devsecops-secrets-lab> docker images
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
devsecops-secrets-lab:latest         b77903632f05                      186MB       45.4MB
PS C:\Users\samue\Desktop\Nova pasta\devsecops-secrets-lab> docker run --rm `
>> -e DATABASE_USER="usuario_teste" `
>> -e DATABASE_PASSWORD="senha_teste" `
>> -e API_TOKEN="token_teste" `
>> devsecops-secrets-lab
Iniciando aplicação...
Usuário do banco carregado: usuario_teste
Senha do banco carregada: ***
Token da API carregado: ***
PS C:\Users\samue\Desktop\Nova pasta\devsecops-secrets-lab>
```

14. Matriz de riscos

Item analisado	Risco identificado	Correção aplicada	Situação final
Código-fonte	Secrets hardcoded	Uso de variáveis de ambiente	Protegido
Arquivo .env	Exposição de credenciais	Exclusão via .gitignore	Protegido
.gitignore	Arquivos sensíveis poderiam ser versionados	Configuração adequada	Protegido
Dockerfile	Secrets poderiam ser incorporados à imagem	Remoção do .env do build	Protegido
.dockerignore	Secrets enviados ao contexto de build	Configuração adequada	Protegido
GitHub Actions	Secrets expostos no workflow	Uso de GitHub Secrets	Protegido
Logs do pipeline	Exposição de credenciais	Mascaramento dos valores	Protegido
GitHub Secrets	Ausência de gerenciamento centralizado	Cadastro dos secrets	Protegido
Imagem Docker	Secrets embarcados na imagem	Uso de variáveis em runtime	Protegido

15. Proposta de hardening

- Não armazenar **secrets** diretamente no código-fonte.
- Utilizar GitHub Secrets para armazenamento seguro.
- Evitar impressão de credenciais nos logs.
- Utilizar .gitignore e .dockerignore adequadamente.
- Realizar rotação periódica de credenciais.
- Implementar secret scanning em repositórios.
- Utilizar Vault ou Secret Manager em ambientes produtivos.
- Revogar imediatamente **secrets** expostos.
- Manter inventário atualizado de credenciais utilizadas pelas aplicações.

16. Conclusão

A atividade demonstrou a aplicação de boas práticas de segurança para proteção de **secrets** em ambientes DevSecOps. Foram implementados mecanismos para impedir a exposição de credenciais no código-fonte, nos logs do pipeline, nos arquivos versionados e nas imagens Docker. Ao final dos testes, o pipeline executou com sucesso e a aplicação utilizou secrets fornecidos de forma segura por variáveis de ambiente.